

JavaScript Identifiers - Complete Reference

What Is an Identifier?

An identifier is a name you choose for variables, constants, functions, classes, object properties, and labels in JavaScript. Choosing valid and meaningful identifiers is essential for writing clean, maintainable code.

1. Core Syntax Rules

JavaScript identifiers must follow these strict rules. Breaking any of them causes a `SyntaxError`.

Rule 1: Allowed Characters

Identifiers can contain letters (A-Z, a-z), digits (0-9), underscore (`_`), and dollar sign (`$`). JavaScript also supports Unicode letters (e.g., e with acute, Chinese characters, Greek lambda).

Valid:

```
let userName = "Alice";
let _count = 0;
let $button = document.querySelector('button');
let total2 = 100;
let unicodeVar = 42;    // Unicode letters are allowed
```

Invalid:

```
let user-name = 5;    // Hyphen not allowed (parsed as subtraction)
let user name = 5;    // Space not allowed
let user@name = 5;    // @ is not allowed
let #tag = 5;         // # is not allowed
```

Rule 2: Cannot Start with a Digit

The first character must be a letter, underscore, or dollar sign. Starting with a digit is illegal.

Valid:

```
let value2 = 10;
let _2nd = 20;
let $2nd = 30;
let secondValue = 40;
```

Invalid:

```
let 2ndValue = 10;    // SyntaxError: Invalid or unexpected token
let 123abc = 5;       // SyntaxError
```

Rule 3: Case Sensitivity

JavaScript identifiers are case-sensitive. `userName`, `UserName`, and `USERNAME` are three completely different identifiers.

Valid:

```
let userName = 'Alice';
let UserName = 'Bob';
let USERNAME = 'Charlie';
console.log(userName); // 'Alice'
console.log(UserName); // 'Bob'
```

Invalid:

```
// There is no error here, but confusing names cause bugs
let data = 1;
let Data = 2;
let DATA = 3;
// Always use distinct, meaningful names instead.
```

JavaScript Identifiers - Complete Reference

Rule 4: No Reserved Keywords

You cannot use JavaScript reserved words as identifiers. Some are always reserved; others are reserved in strict mode or specific contexts.

Valid:

```
let myClass = 'Economics'; // 'class' is reserved, but 'myClass' is fine
let _if = true;             // Underscore prefix makes it valid
let $return = 5;           // Dollar prefix makes it valid
```

Invalid:

```
let class = 10;             // SyntaxError: Unexpected token 'class'
let return = 5;             // SyntaxError: Unexpected token 'return'
let if = true;              // SyntaxError: Unexpected token 'if'
let const = 1;              // SyntaxError: Unexpected token 'const'
```

JavaScript Identifiers - Complete Reference

2. Reserved Words (Forbidden Identifiers)

The following words cannot be used as identifiers in any context. Using them will produce a `SyntaxError` immediately.

Always Reserved (ECMAScript standard):

<code>break</code>	<code>case</code>	<code>catch</code>	<code>class</code>	<code>const</code>
<code>continue</code>	<code>do</code>	<code>do</code>	<code>class</code>	<code>instanceof</code>
<code>function</code>	<code>else</code>	<code>for</code>	<code>finally</code>	<code>var</code>
<code>void</code>	<code>while</code>	<code>with</code>	<code>yield</code>	

Reserved in Strict Mode or Specific Contexts:

<code>enum</code>	<code>await</code>	<code>implements</code>	<code>package</code>	<code>protected</code>
<code>export</code>	<code>interface</code>	<code>private</code>	<code>private</code>	<code>private</code>
<code>short</code>	<code>synchronized</code>	<code>throw</code>	<code>transient</code>	<code>volatile</code>

Note: In strict mode, some additional words like 'let' and 'static' are also enforced. Modern modules and classes implicitly run in strict mode.

JavaScript Identifiers - Complete Reference

3. Naming Conventions (Best Practices)

While JavaScript allows many identifier styles, the community follows strong conventions for readability and consistency.

camelCase

Usage: Variables, functions, methods, and object properties

```
let userName = 'Alice';
function getUserData() { ... }
const calculateTotal = () => ...;
let firstName = 'John';
```

PascalCase

Usage: Classes, constructors, and React/Vue components

```
class UserProfile { ... }
class ShoppingCart { ... }
function MyComponent() { ... }
class DataProcessor { ... }
```

SCREAMING_SNAKE_CASE

Usage: Constants and configuration values

```
const MAX_SIZE = 100;
const API_URL = 'https://api.example.com';
const DEFAULT_TIMEOUT = 5000;
const PI = 3.14159;
```

snake_case

Usage: Sometimes used in object keys or legacy code; less common in modern JS

```
let user_name = 'Alice'; // Valid but not preferred
const api_key = 'abc';    // Valid but camelCase is preferred
```

JavaScript Identifiers - Complete Reference

4. Special Prefix Meanings

While not enforced by the language, certain prefixes carry conventional meaning in JavaScript codebases.

Underscore (_) prefix

Often indicates a 'private' or internal member. It is a convention only - JavaScript does not enforce true privacy with a single underscore.

```
class Counter {  
  _count = 0;           // intended as private  
  _increment() { ... } // intended as private  
}
```

Double underscore (__) prefix

Rarely used in user code. Some frameworks use it for internal utilities. Avoid using this in your own identifiers to prevent confusion.

```
// Example from frameworks (not user code)  
// React uses __SECRET_INTERNALS_DO_NOT_USE_OR_YOU_WILL_BE_FIRED
```

Dollar sign (\$) prefix

Historically associated with jQuery or DOM element references. Still common in code that manipulates DOM nodes directly.

```
const $button = document.querySelector('#submit');  
const $input = document.getElementById('name');  
const $modal = $('<div>.modal'); // jQuery-style
```

JavaScript Identifiers - Complete Reference

5. Complete Valid vs Invalid Examples

The table below covers every common scenario you will encounter when naming identifiers in JavaScript.

Identifier	Status	Explanation
<code>userName</code>	Valid	Standard camelCase variable
<code>UserName</code>	Valid	PascalCase variable (allowed but usually for classes)
<code>user_name</code>	Valid	snake_case is syntactically valid
<code>_count</code>	Valid	Underscore prefix, internal-use convention
<code>\$button</code>	Valid	Dollar prefix, DOM element convention
<code>value2</code>	Valid	Digits allowed after first character
<code>_2nd</code>	Valid	Starts with underscore, contains digit
<code>\$2nd</code>	Valid	Starts with dollar, contains digit
<code>pi</code>	Valid	Unicode letter (Greek pi) is valid
<code>unicodeVar</code>	Valid	Unicode letter (Chinese) is valid
<code>lambda</code>	Valid	Unicode letter (Greek lambda) is valid
<code>__proto__</code>	Valid	Double underscore, special but valid
<code>2ndValue</code>	Invalid	Cannot start with a digit
<code>123abc</code>	Invalid	Cannot start with a digit
<code>user-name</code>	Invalid	Hyphen is not allowed
<code>user name</code>	Invalid	Space is not allowed
<code>user@name</code>	Invalid	@ is not allowed
<code>#tag</code>	Invalid	# is not allowed
<code>user.name</code>	Invalid	Dot is not allowed in a simple identifier
<code>class</code>	Invalid	Reserved keyword
<code>return</code>	Invalid	Reserved keyword
<code>if</code>	Invalid	Reserved keyword
<code>const</code>	Invalid	Reserved keyword
<code>let</code>	Invalid	Reserved keyword
<code>function</code>	Invalid	Reserved keyword
<code>true</code>	Invalid	Reserved literal (boolean)
<code>false</code>	Invalid	Reserved literal (boolean)
<code>null</code>	Invalid	Reserved literal
<code>undefined</code>	Valid	Not a reserved word, but avoid using as variable name
<code>NaN</code>	Valid	Global property, not a reserved word, but avoid reassigning
<code>Infinity</code>	Valid	Global property, not a reserved word, but avoid reassigning

Note: Even if an identifier is syntactically valid, it may be a bad practice to use it. For example, reassigning 'undefined' or 'NaN' can cause subtle bugs even though the parser allows it.

JavaScript Identifiers - Complete Reference

6. Common Mistakes & How to Fix Them

Using hyphens in variable names

JavaScript interprets hyphens as the subtraction operator.

Wrong:

```
let user-name = 'Alice'; // SyntaxError
```

Right:

```
let userName = 'Alice'; // Correct
```

Starting with a number

The parser expects an identifier to start with a non-digit character.

Wrong:

```
let 2ndPlace = 'silver'; // SyntaxError
```

Right:

```
let secondPlace = 'silver'; // Correct
let _2ndPlace = 'silver'; // Also correct
```

Using spaces inside identifiers

Spaces separate tokens in JavaScript.

Wrong:

```
let user name = 'Alice'; // SyntaxError
```

Right:

```
let userName = 'Alice'; // Correct
let user_name = 'Alice'; // Also correct
```

Shadowing reserved words without meaning to

Accidentally naming a variable 'class' or 'let' inside a function.

Wrong:

```
function demo() { let class = 'A'; } // SyntaxError
```

Right:

```
function demo() { let className = 'A'; } // Correct
```

Case confusion when importing/exporting

A module exports 'myData' but you import 'MyData'.

Wrong:

```
// Module A: export const myData = 1;
// Module B: import { MyData } from './A'; // Error: MyData not found
```

Right:

```
// Module B: import { myData } from './A'; // Correct
```

JavaScript Identifiers - Complete Reference

7. Quick Reference Checklist

Before you use an identifier, run through this checklist:

- ☐ Does it start with a letter, underscore (`_`), or dollar sign (`$`)?
- ☐ Does it contain only letters, digits, underscore, or dollar sign?
- ☐ Is it NOT a reserved keyword (e.g., `let`, `class`, `return`, `if`)?
- ☐ Is it descriptive and meaningful (e.g., `totalPrice` instead of `x`)?
- ☐ Does it follow the project's naming convention (`camelCase` / `PascalCase` / `SCREAMING_SNAKE_CASE`)?
- ☐ Is it not too similar to another identifier in the same scope (case-sensitive)?
- ☐ Are you avoiding reassigning global properties like `undefined`, `NaN`, or `Infinity`?

Summary

Valid identifiers in JavaScript start with a letter, underscore, or dollar sign, contain only allowed characters, avoid reserved words, and follow consistent naming conventions. When in doubt, use `camelCase` for variables and functions, `PascalCase` for classes, and `SCREAMING_SNAKE_CASE` for constants.